

Engenharia de Software III

Bem vindos à Engenharia de
Software III

Profa. Renata Neves

Engenharia de Software III

- ✓ Arquitetura orientada a serviços (SOA – service Oriented Architecture)
- ✓ Arquitetura Orientada a Eventos (EDA – *Event Driven Architecture*)
- ✓ Barramento de serviços – ESB (Enterprise Service Bus)



Engenharia de Software III

SOA

“um estilo de arquitetura onde funcionalidades de aplicações de negócio existentes (LoB – Line of Business) são disponibilizadas e publicadas na forma de serviços”.

Importante o mapeamento dos serviços e funcionalidades disponíveis, que irão atender as necessidades do negócio.

Deve estar alinhado com o planejamento estratégico da empresa;

Utilizado para integrar aplicações diferentes dentro de um ERP, CRM, integração externa, pagamentos por diversas plataformas de cartão de crédito, por exemplo.

Engenharia de Software III

O grupo de Arquitetura do W3C define SOA como uma forma de arquitetura de sistemas distribuídos, tipicamente caracterizada pelas propriedades indicadas:

Propriedade	Descrição
Visão lógica	O serviço é uma abstração, uma visão lógica de programas reais, bases de dados, processos de negócios e assim por diante, definidos em termos do que ele faz, geralmente realizando uma operação de nível empresarial. Serviço é definido como uma ação de negócios significativo.
Orientado à mensagens	O serviço é definido formalmente em termos das mensagens trocadas entre provedor e consumidor. A estrutura interna da implementação é deliberadamente abstraída. Interface de serviço é separada da implementação do serviço.

Engenharia de Software III

Orientado à descrição	o serviço é descrito por metadados processáveis / definição de serviço.
Granularidade	Serviços tendem a usar pequeno número de operações com mensagens complexas e extensas
Orientado à redes	Uso sobre redes
Plataforma neutra	As mensagens são enviadas em um formato padronizado de plataforma neutra, entregues através das interfaces. O mais óbvio é o formato XML, que atende essa restrição.

Engenharia de Software III

Algumas conclusões:

- **SOA não é uma tecnologia.** Há tanto de negócio quanto de tecnologia em SOA. As tecnologias (padrões) que dão suporte a SOA são o que a viabiliza, mas SOA não é uma tecnologia por si só.
- **SOA não é uma metodologia.** Há várias metodologias (processos, ferramentas, métodos de trabalho) que podem ser usados para implantar SOA com sucesso. SOA **não é** e **nem define** alguma metodologia.
- **SOA pode ser considerada uma filosofia arquitetural.** SOA é uma linha de pensamento que permeia a implementação de necessidades de negócio, refletida em diretrizes, políticas e metodologias corporativas, não necessariamente restritas à área de TI.
- **SOA não é algo que se possa comprar ou instalar.**
- **SOA não é um webservice.**
- **SOA não cria nada.** Ela apenas sugere, propõe, define.

Engenharia de Software III

No livro, “SOA: Princípios do Design de Serviços”, Thomas Erl, ele concretizou as 8 “regras” para definir um serviço, e são elas:

1. Serviços são CAPAZES DE SE COMPOR

Separar um grande problema em vários pequenos não é uma novidade. Este mesmo princípio pode ser utilizado na construção dos Serviços. Assim, um Serviço pode “chamar” outro(s) para executar a sua tarefa. A composição também é uma forma de reutilização.

2. Serviços são AUTÔNOMOS

Autonomia significa a capacidade de se auto-governar. Um Serviço autônomo é aquele que independe de um elemento externo para executar sua lógica.

Engenharia de Software III

3. Serviços evitam ALOCAÇÃO DE RECURSOS por longos períodos

Os serviços evitam informação de Estado.

Devido ao fato de que um Serviço será reutilizado, deve-se tomar o cuidado de não se criar muitas instâncias deste Serviço onerando a infra-estrutura. Isto significa que para um Serviço ter disponibilidade, ele deve evitar reter informações específicas a uma determinada atividade.

4. Serviços devem possuir a CAPACIDADE DE SEREM DESCOBERTOS

Um contrato formal bem descrito e padronizado evita que novos requerimentos resultem em Serviços redundantes. Além disto a arquitetura deve prover mecanismos que facilitem o descobrimento dos Serviços através de Diretórios e Registros.

Engenharia de Software III

5. Serviços são reutilizáveis;

permite que a área de TI forneça respostas rápidas a novos requerimentos das áreas de negócio da empresa, desde que estes Serviços sejam definidos em conjunto: TI e área de negócios.

6. Serviços compartilham um CONTRATO FORMAL

Todo serviço possui um “contrato” entre o requisitante e o provedor deste serviço. O “contrato”, informa o que o Serviço faz e como se comunica (o que deve receber e o que deve entregar).

Engenharia de Software III

7. Serviços possuem BAIXO ACOPLAMENTO

O acoplamento se refere a uma medida de dependência.

Um baixo acoplamento significa que implementações específicas de um Serviço podem ser substituídas, modificadas e evoluídas sem que os consumidores deste Serviço sintam qualquer descontinuidade.

8. Serviços ABSTRAEM A LÓGICA

Serviços são como “caixas pretas”, o que significa que a lógica não precisa nem deve ser exposta, simplificando o contrato formal.

Engenharia de Software III

Serviços

- ✓ serviços são autônomos;
- ✓ serviços são orientados a mensagens;
- ✓ serviços podem suportar diferentes protocolos e mecanismos de transporte;
- ✓ serviços podem ser publicados ou hospedados em diferentes tipos de hosts;
- ✓ serviços suportam contratos de operação, interfaces e tipos de mensagens;"

Engenharia de Software III

Serviços comuns de uma arquitetura SOA

- ✓ Apresentação dos serviços;
- ✓ Colaboração dos serviços;
- ✓ Interação dos serviços;
- ✓ Processamento dos serviços;
- ✓ Dados dos serviços;
- ✓ Conectividade dos serviços;
- ✓ Monitoramento e segurança dos serviços;
- ✓ Orquestração dos serviços.

Engenharia de Software III

APIS - comuns de uma arquitetura SOA

Chamadas de Interfaces de Programação de Aplicativos, as APIs (na sigla em inglês), possibilitam a integração de tecnologias que operam a base de um protocolo HTTP. Seu objetivo é promover a comunicação entre um software e outras aplicações que precisam se integrar.

A estratégia adotada para criação e utilização das APIs, são importantes para flexibilizar e entregar rapidamente valor ao negócio, conectando aplicações internas, clientes, parceiros de negócio e novas funcionalidades.

As APIs costumam ser o meio pelo qual os microsserviços se comunicam. Além disto, podem ser usadas para expor os dados e as funcionalidades de uma aplicação a terceiros, abrindo caminho para integrações poderosas.

Engenharia de Software III

APIS - podem ser:

Privada – são desenhadas para integrar processos internos do negócio, servindo de suporte para uma estratégia de arquitetura de microserviços. São utilizadas para integração de sistemas internos, desenvolvidos separadamente;

Restrita – são expostas apenas a parceiros do negócio. Assim, cria-se um ecossistema de desenvolvedores envolvidos em desenhar produtos e serviços inovadores a partir dos dados disponibilizados.

Pública – são disponibilizadas abertamente para que qualquer desenvolvedor possa utilizar os dados no *backend* da plataforma como suporte para suas aplicações.

Engenharia de Software III

APIS

Exemplos de APIs :

- ✓ APIs que fazem parte das funcionalidades de um website, buscadores de viagens, hotéis, certificadores, autorizadores de cartões de crédito;
- ✓ APIs que são utilizadas em softwares online ([SaaS](#));
- ✓ APIs suportam as aplicações móveis (Apps Mobile);
- ✓ APIs e Internet das Coisas tem relação direta.

<https://www.programmableweb.com/category/all/news?articletypes=research>

Engenharia de Software III

Arquitetura Orientada a Eventos (EDA – *Event Driven Architecture*)

Evento?

- ✓ Um evento caracteriza uma ocorrência de um fato, dentro de um sistema de uma organização;
- ✓ Possibilita uma comunicação organizada e controlada entre sistemas;
- ✓ Definem o momento da integração;
- ✓ Carregam informações;
- ✓ Relevante para o processo de negócio da empresa;

Exemplos: Cadastro de um correntista, desistência de uma compra.

Engenharia de Software III

Arquitetura Orientada a Eventos (EDA – *Event Driven Architecture*)

Evento?

A definição do evento, está muito mais ligado ao processo de negócio da empresa, do que a decisões técnicas. Mas podemos definir eventos técnicos.

Ex.: a quantidade limite para execução de um serviço disponível, o tempo limite de resposta de uma transação.

Engenharia de Software III

Arquitetura Orientada a Eventos (EDA – *Event Driven Architecture*)

Evento?

Um evento é autocontido, isto é, carrega consigo a informação de quando está preparado para ser disparado e quais informações deve conter.

Engenharia de Software III

Arquitetura Orientada a Eventos (EDA – *Event Driven Architecture*)

Arquitetura Orientada a Eventos (EDA – *Event Driven Architecture*) promove uma estrutura que possa estabelecer o alicerce de apoio à integração entre os sistemas e/ou componentes do ambiente – usufruindo da ocorrência de evento, de forma que ainda se mantenha um baixo acoplamento entre os sistemas.

Engenharia de Software III

Arquitetura Orientada a Eventos (EDA – *Event Driven Architecture*)

Assim como fazemos a modelagem da análise de domínio, devemos também analisar como os eventos serão tratados para a solução do negócio através de sua aplicação.

Dentro dos vários sistemas da empresa, existem vários emissores de eventos, e assinantes de eventos.

Uma arquitetura orientada a eventos consiste em produtores de eventos que geram um fluxo de eventos e consumidores de eventos que ouvem os eventos.

Engenharia de Software III

Arquitetura Orientada a Eventos (EDA – *Event Driven Architecture*)

Os eventos trabalham em tempo real, são respondidos à medida que ocorrem, os produtores são dissociados dos consumidores - um produtor não sabe quais consumidores estão ouvindo.

Os consumidores também são dissociados uns dos outros, e todo consumidor vê todos os eventos. Em lot são ingeridos em volumes muito altos.

Engenharia de Software III

Arquitetura Orientada a Eventos (EDA – *Event Driven Architecture*)

Modelos: modelo de publicação / assinatura
publication / subscription (**Pub** / **sub**)

A infra-estrutura de mensagens acompanha as inscrições. Quando um evento é publicado, ele envia o evento para cada assinante. Depois que um evento é recebido, ele não pode ser repetido e novos inscritos não veem o evento.

Engenharia de Software III

Arquitetura Orientada a Eventos (EDA – *Event Driven Architecture*)

Modelos: fluxo de eventos

Streaming de eventos: os eventos são gravados em um log. Eventos são estritamente ordenados (dentro de uma partição) e duráveis. Os clientes não assinam o fluxo, em vez disso, um cliente pode ler de qualquer parte do fluxo. O cliente é responsável por avançar sua posição no fluxo. Isso significa que um cliente pode participar a qualquer momento e pode reproduzir eventos.

Engenharia de Software III

Arquitetura Orientada a Eventos (EDA – *Event Driven Architecture*)

A origem dos eventos pode ser externa ao sistema, como dispositivos físicos em uma solução de IoT. Nesse caso, o sistema deve ser capaz de ingerir os dados no volume e na taxa de transferência exigidos pela fonte de dados.

Engenharia de Software III

Arquitetura Orientada a Eventos (EDA – *Event Driven Architecture*)

Para melhorar a organização e o controle da estrutura da EDA, utilizamos um barramento de serviços ESB (Enterprise Service Bus), como um mediador.

Ao termos um mediador responsável por tratar as integrações, que centraliza os serviços existentes no ambiente, criar uma arquitetura para eventos se torna mais fácil.

Engenharia de Software III

Arquitetura Orientada a Eventos (EDA – *Event Driven Architecture*)

ESB (Enterprise Service Bus)

Pelo ESB (Enterprise Service Bus), barramento de serviços podemos identificar os serviços expostos de sistemas que são potenciais emissores (origem de eventos), deixando a responsabilidade de gerenciamento dos eventos ao barramento de serviços.

Engenharia de Software III

Arquitetura Orientada a Eventos (EDA – *Event Driven Architecture*)

ESB (Enterprise Service Bus)

O Barramento de Serviço é mais comumente usado para desacoplar aplicativos e serviços uns dos outros e é uma plataforma confiável e segura para dados assíncronos e transferência de estado. Os dados são transferidos entre diferentes aplicativos e serviços usando *mensagens*. Uma mensagem é em formato binário que pode conter JSON, XML ou apenas texto.

<https://docs.microsoft.com/pt-br/azure/service-bus-messaging/service-bus-messaging-overview>

Engenharia de Software III

Arquitetura Orientada a Eventos (EDA – *Event Driven Architecture*)

Alguns cenários de sistema de mensagens comuns são:

- ✓ Mensagens: transferir dados comerciais, como vendas ou pedidos de compra, diários ou movimentos de estoque.
- ✓ Desacoplar aplicativos: melhorar a confiabilidade e a escalabilidade de aplicativos e serviços (cliente e serviço não precisam estar online ao mesmo tempo).
- ✓ Tópicos e assinaturas: habilitar 1: n relacionamentos entre publicadores e assinantes.
- ✓ Sessões de mensagens: implementar fluxos de trabalho que exijam ordenação de mensagens ou adiamento de mensagens.

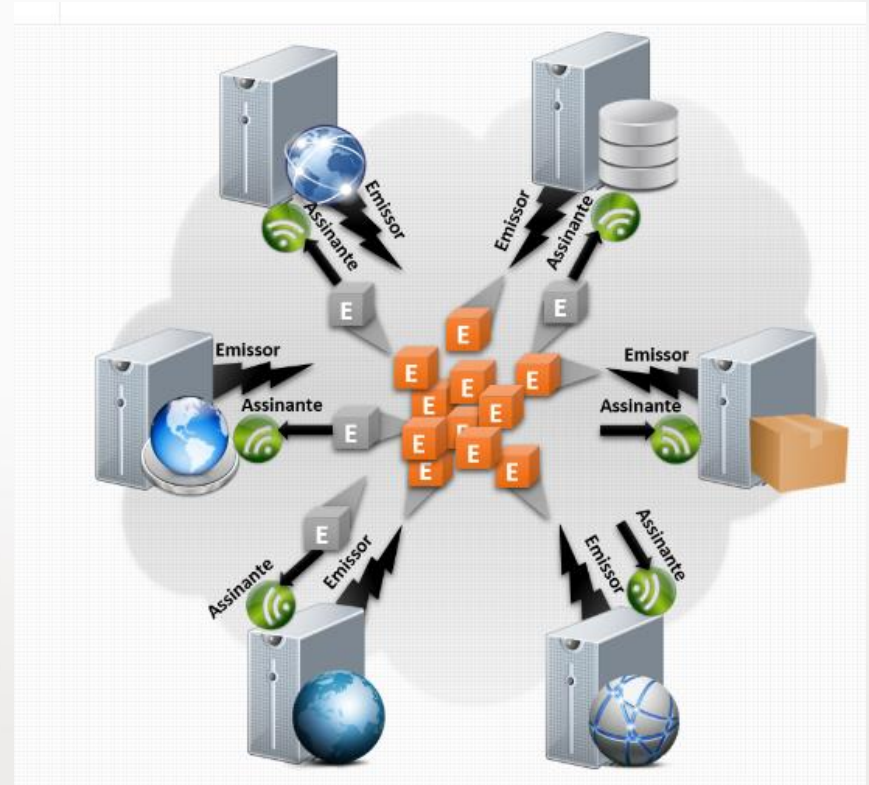
<https://docs.microsoft.com/pt-br/azure/service-bus-messaging/service-bus-messaging-overview>

Engenharia de Software III

Arquitetura Orientada a Eventos (EDA – *Event Driven Architecture*) ESB (Enterprise Service Bus)

O descontrolado sobre a integração acaba se espalhando, resultando em: eventos duplicados, eventos específicos por sistema, desconhecimento de assinantes e acoplamento mais alto entre as partes.

*Na **Figura 1** podemos analisar uma ilustração de como seria um ambiente previamente concebido estabelecendo uma arquitetura orientada a eventos sem nenhuma organização e controle sobre as integrações.*



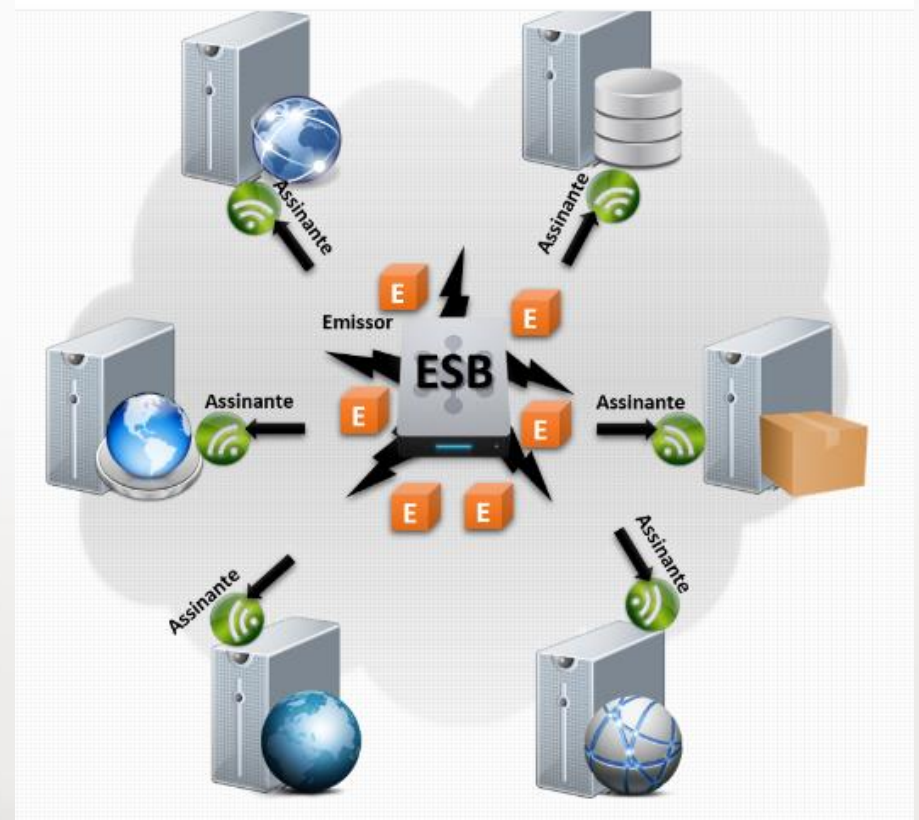
Fonte: <https://www.devmedia.com.br>

Engenharia de Software III

Arquitetura Orientada a Eventos (EDA – *Event Driven Architecture*) ESB (Enterprise Service Bus)

Já na Figura 2 podemos observar uma ilustração de uma Arquitetura Orientada a Eventos funcionando com o apoio de um barramento de serviços de uma arquitetura SOA. Neste cenário, o barramento é responsável por centralizar a emissão dos eventos do ambiente, realizando a inspeção do fluxo de informações que está trafegando durante a execução dos serviços expostos pelos sistemas.

Assim que identificado e criado um evento pelo barramento, os assinantes podem, em seguida, recebê-lo.



Fonte: <https://www.devmedia.com.br>

Engenharia de Software III

Arquitetura Orientada a Eventos (EDA – *Event Driven Architecture*)



.NET



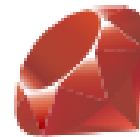
Java



Node.js



Python



Ruby

Engenharia de Software III

Quando usar essa arquitetura?

- ✓ Vários subsistemas devem processar os mesmos eventos.
- ✓ Processamento em tempo real com atraso de tempo mínimo.
- ✓ Processamento de eventos complexos, como janelas de correspondência de padrões ou agregação ao longo do tempo.
- ✓ Alto volume e alta velocidade de dados, como IoT.

<https://docs.microsoft.com/pt-br/azure/architecture/guide/architecture-styles/event-driven>

Engenharia de Software III

Benefícios

- ✓ Produtores e consumidores são dissociados.
- ✓ Nenhuma integração ponto a ponto. É fácil adicionar novos consumidores ao sistema.
- ✓ Os consumidores podem responder aos eventos imediatamente à medida que chegam.
- ✓ Altamente escalável e distribuído.
- ✓ Os subsistemas têm visualizações independentes do fluxo de eventos.

<https://docs.microsoft.com/pt-br/azure/architecture/guide/architecture-styles/event-driven>

Engenharia de Software III

Tp, pesquisa:

Quais as principais responsabilidades e funcionalidades que um ESB pode fornecer para Organizar uma arquitetura EDA.

Incluir nome da ferramenta para as seguintes empresas:

- Azure;
- AWS – Amazon Web service;
- Oracle;
- IBM;
- Google.

Engenharia de Software III



Fonte: <http://www.w3schools.com>